EliteDesign: Elite-Guided Learning for RNA Design via the Cross-Entropy Method

JasonFred Ngwa[1] Milan Gautam[1] David H. Mathews[2] Liang Huang[1,†]

[1]School of EECS, Oregon State University, Corvallis, OR 97331, USA
[2]Dept. of Biochemistry & Biophysics, University of Rochester Medical Center, Rochester, NY 14642, USA
[†]Corresponding author

Abstract

The goal of RNA design is to find, within an exponentially large sequence space, a nucleotide sequence that folds into a specified target structure. While essential for developing functional RNAs such as riboswitches and ribozymes, efficiently identifying sequences with high Boltzmann probability remains challenging, especially for long sequences. Currently, the state-of-the-art method, SamplingDesign, is computationally very expensive, because its gradient-based optimization takes many small steps and scores thousands of sequences at each step. To address this bottleneck, we present EliteDesign, a novel framework that leverages the gradient-free cross-entropy method to iteratively refine a distribution over candidate RNA sequences. On the Eterna100 benchmark, EliteDesign achieves a higher average Boltzmann probability than SamplingDesign while running 178$\times$ faster. Furthermore, EliteDesign scales efficiently to long RNA structures far beyond the reach of SamplingDesign. On ten 16S rRNA targets (950–1,525 nucleotides), it achieves an average Boltzmann probability of 0.52 versus 0.20 for the strongest scalable baseline, SAMFEO, while accelerating runtime by 14$\times$.

Keywords: RNA design; cross-entropy method; gradient-free optimization; ensemble-based optimization; computational efficiency

1 Introduction

RNA plays two main roles in cells. It can serve as an information carrier, carrying the genetic code needed to make proteins, but it can also serve a functional role such as catalyzing chemical reactions or regulating genes (Breaker, 2012; Doudna and Cech, 2002). Recent advances have shown that RNA has huge potential for developing new medicines. For example, Rojas et al. (2023) showed that personalized mRNA vaccines stimulated immune responses against tumor targets in patients with pancreatic cancer. This advancement could open the path toward cancer treatments that are tailored to individual patients. Additionally, on the computational side, recent methods such as LinearDesign have shown that algorithmic techniques can be used to design mRNAs with improved stability, protein production, and immune response (Zhang et al., 2023). These advances in RNA technology motivate further work on designing RNA.

RNA design, also called inverse folding, is the task of finding a nucleotide sequence that folds into a given target structure (Dirks et al., 2004; Hofacker et al., 1994). Given a target structure, a good RNA design method should find a sequence that favors the target over the competing folds in its ensemble. However, this problem is difficult because the number of possible sequences for a target grows exponentially with length, and each of these sequences can adopt many alternative structures, which makes the search especially hard for long RNAs. Existing methods that optimize Boltzmann probability face limitations in quality, runtime, or both. For example, SAMFEO maintains several promising sequences and uses structure-aware mutations to optimize Boltzmann probability (Zhou et al., 2023), but it struggles on difficult and long targets, where the enormous design space makes local search prone to getting stuck in local optima. On the other hand, SamplingDesign (Tang et al., 2026) produces high-quality designs, even for difficult puzzles in Eterna100, by running gradient descent on a distribution over the valid sequences, but it is computationally expensive and too slow to be practical for longer RNAs. Finding a high-probability sequence within a practical runtime remains a major challenge.

To address this bottleneck, we present EliteDesign, a gradient-free method that uses the cross-entropy method (de Boer et al., 2005; Rubinstein, 1999) to find RNA sequences with high Boltzmann probability in a computationally efficient way. At each iteration, EliteDesign samples from the valid design space, scores the sequences, and then picks a set of elite sequences based on Boltzmann probability. It then moves the current distribution toward the distribution described by the elite set. During the sampling rounds, sequences are scored with LinearPartition (Zhang et al., 2020), which estimates Boltzmann probability in linear time, and the best final candidates are rescored exactly with ViennaRNA (Lorenz et al., 2011).

We benchmark EliteDesign on several datasets and show that it finds higher Boltzmann probabilities than the methods we compare against, with the gap widening on long RNA sequences. These results show that EliteDesign improves both design quality and runtime compared with methods that optimize Boltzmann probability, and allows us to find high-probability sequences for much longer RNA structures.

2 RNA Design Problem

2.1 Valid design space

A sequence $\boldsymbol{x}$ of length n is a string $x_1 x_2 \dots x_n$ over the nucleotide alphabet $\mathcal{N} \triangleq \{\mathtt{A}, \mathtt{C}, \mathtt{G}, \mathtt{U}\}$. We describe a pseudoknot-free secondary structure $\boldsymbol{y}$ by a balanced dot-bracket string of the same length, where each dot marks an unpaired nucleotide and each matching pair of brackets marks a base pair. The set $\mathit{unpaired}(\boldsymbol{y})$ collects the unpaired positions and the set $\mathit{pairs}(\boldsymbol{y})$ collects the base pairs. For example, consider the structure in Fig. 1, shown here with its position numbers.

$$\boldsymbol{y} = \begin{array}{cccccccccc} (& . & (& . & . & . &) & . & . &) \\ 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 & 10 \end{array}$$

Position 1 pairs with position 10, and position 3 pairs with position 7, so $\mathit{pairs}(\boldsymbol{y}) = \{(1, 10), (3, 7)\}$. The remaining positions are unpaired, so $\mathit{unpaired}(\boldsymbol{y}) = \{2, 4, 5, 6, 8, 9\}$.

RNA design starts from a target structure, which we write as $\boldsymbol{y}^\star$, and asks for a sequence that folds into it. Not every sequence is a candidate, because the two nucleotides of a target pair must be able to pair with each other, as a Watson–Crick–Franklin or wobble pair from $\mathcal{P} \triangleq \{\mathtt{CG}, \mathtt{GC}, \mathtt{AU}, \mathtt{UA}, \mathtt{GU}, \mathtt{UG}\}$. We therefore search only the valid design space,

$$\mathcal{X}(\boldsymbol{y}^\star) \triangleq \{\boldsymbol{x} \in \mathcal{N}^{|\boldsymbol{y}^\star|} \mid \forall (i,j) \in \mathit{pairs}(\boldsymbol{y}^\star), \; x_i x_j \in \mathcal{P}\}, \tag{1}$$

which contains

$$|\mathcal{X}(\boldsymbol{y}^\star)| = 4^{|\mathit{unpaired}(\boldsymbol{y}^\star)|} \cdot 6^{|\mathit{pairs}(\boldsymbol{y}^\star)|} \ll 4^{|\boldsymbol{y}^\star|} \tag{2}$$

sequences. After ruling out incompatible choices, such as C pairing with A, the search space shrinks by a great deal. However, the number of sequences still grows exponentially as the target gets longer, so enumerating the whole design space is impractical.

2.2 Boltzmann probability

Now that we have a valid design space for the target, we need a way to assess how well a given sequence favors folding into it. To do this, we calculate the Boltzmann probability, which is the probability that the sequence folds into the target, given its competing structures.

Let $\mathcal{Y}(\boldsymbol{x})$ be the ensemble of $\boldsymbol{x}$, meaning every secondary structure that $\boldsymbol{x}$ can adopt. Under the Turner 2004 nearest-neighbor model (Mathews et al., 2004), each structure $\boldsymbol{y} \in \mathcal{Y}(\boldsymbol{x})$ has a Gibbs free energy change $\Delta G^\circ(\boldsymbol{x}, \boldsymbol{y})$, and lower values mean greater stability. The partition function adds up a Boltzmann factor for each structure in the ensemble (McCaskill, 1990).

$$Q(\boldsymbol{x}) \triangleq \sum_{\boldsymbol{y} \in \mathcal{Y}(\boldsymbol{x})} e^{-\Delta G^\circ(\boldsymbol{x}, \boldsymbol{y})/RT}, \tag{3}$$

where T is the absolute temperature and R is the molar gas constant. Structures with lower free energy contribute more to this sum. The Boltzmann probability of a structure $\boldsymbol{y}$ is its share of the sum.

$$p(\boldsymbol{y} \mid \boldsymbol{x}) = \frac{e^{-\Delta G^\circ(\boldsymbol{x}, \boldsymbol{y})/RT}}{Q(\boldsymbol{x})}. \tag{4}$$

2.3 Normalized Ensemble Defect

Boltzmann probability only rewards the exact target, so a competing structure that misses by one nucleotide counts the same as one that looks nothing like the target. Normalized Ensemble Defect (NED) grades these competitors instead. It is the expected fraction of nucleotides whose pairing state differs from the target (Dirks et al., 2004; Zadeh et al., 2011).

To compute it, let p_{ij} be the total probability of the ensemble structures that contain the pair (i, j), and let q_i be the probability that nucleotide i is unpaired. The positional defect $\epsilon_i(\boldsymbol{x}, \boldsymbol{y}^\star)$ of a nucleotide is the probability that it is not in its target state, which is $1 - q_i$ for an unpaired target position and $1 - p_{ij}$ for a position whose target partner is j. NED is the average of these defects over the sequence.

$$\mathrm{NED}(\boldsymbol{x}, \boldsymbol{y}^\star) \triangleq \frac{1}{|\boldsymbol{x}|} \sum_{i=1}^{|\boldsymbol{x}|} \epsilon_i(\boldsymbol{x}, \boldsymbol{y}^\star). \tag{5}$$

A value of 0 means that every nucleotide is always in its target state.

2.4 MFE metrics

RNA design benchmarks such as Eterna100 are traditionally judged by the lowest energy structure of a sequence, so we report these metrics as well. The following equation gives the minimum free energy (MFE) structure of a sequence $\boldsymbol{x}$, which is the structure in its ensemble with the lowest free energy.

$$\mathrm{MFE}(\boldsymbol{x}) \triangleq \operatorname*{argmin}_{\boldsymbol{y} \in \mathcal{Y}(\boldsymbol{x})} \Delta G^\circ(\boldsymbol{x}, \boldsymbol{y}). \tag{6}$$

Several structures can share the lowest free energy, so we write $\mathrm{MFEs}(\boldsymbol{x})$ for the set of all of them. A sequence $\boldsymbol{x}$ is an MFE solution if the target structure $\boldsymbol{y}^\star$ is in $\mathrm{MFEs}(\boldsymbol{x})$, and a uMFE (unique MFE) solution if $\boldsymbol{y}^\star$ is the only structure in $\mathrm{MFEs}(\boldsymbol{x})$.

We use two metrics to measure how close a sequence is to being an MFE solution. The first is the structural distance, which counts the nucleotides that fold differently in two structures $\boldsymbol{y}$ and $\boldsymbol{y}'$.

$$\begin{aligned} d(\boldsymbol{y}, \boldsymbol{y}') \triangleq\ & |\boldsymbol{y}| - 2 \cdot |\mathit{pairs}(\boldsymbol{y}) \cap \mathit{pairs}(\boldsymbol{y}')| \\ & - |\mathit{unpaired}(\boldsymbol{y}) \cap \mathit{unpaired}(\boldsymbol{y}')|. \end{aligned} \tag{7}$$

We use this equation to compare $\mathrm{MFE}(\boldsymbol{x})$, the MFE structure of $\boldsymbol{x}$, with $\boldsymbol{y}^\star$, the target structure for which we are optimizing. The second metric is the free energy gap. It measures the difference between the free energy of $\boldsymbol{y}^\star$ and the free energy of $\mathrm{MFE}(\boldsymbol{x})$.

$$\begin{aligned} \Delta\Delta G^\circ(\boldsymbol{x}, \boldsymbol{y}^\star) \triangleq\ & \Delta G^\circ(\boldsymbol{x}, \boldsymbol{y}^\star) \\ & - \Delta G^\circ(\boldsymbol{x}, \mathrm{MFE}(\boldsymbol{x})) \geq 0. \end{aligned} \tag{8}$$

A sequence with a structural distance of 0 or a free energy gap of 0 is an MFE solution. A free energy gap of 0 does not make it a uMFE solution, because another structure can have the same free energy as $\boldsymbol{y}^\star$.

2.5 Objective function

During optimization, any of these metrics (Boltzmann probability, NED, structural distance, and free energy gap), or any other relevant metric, can be used as an objective function $f(\boldsymbol{x}, \boldsymbol{y}^\star)$. RNA design is then the search for

$$\boldsymbol{x}^\star = \operatorname*{argmin}_{\boldsymbol{x} \in \mathcal{X}(\boldsymbol{y}^\star)} f(\boldsymbol{x}, \boldsymbol{y}^\star). \tag{9}$$

When Boltzmann probability is used, optimization is typically done in log space to avoid underflow.

3 Continuous Optimization over Sequence Distributions

3.1 Prior work on sequence distributions

Previously, RNA design was mostly approached with local search methods such as SAMFEO (Zhou et al., 2023) and NEMO (Portela, 2018). These methods hold on to a few candidate sequences and improve them through small mutations. However, the design space grows exponentially with length, so these mutations can only reach a tiny part of it.

To address this, Matthies et al. (2023) replaced the candidate sequences with a probability distribution over sequences, which is optimized with gradient descent. SamplingDesign (Tang et al., 2026) then restricted this distribution, which they write as $p_{\boldsymbol{y}}(\cdot)$, to the valid design space $\mathcal{X}(\boldsymbol{y}^\star)$. Their goal is then to minimize the expected value of the objective function $f(\boldsymbol{x}, \boldsymbol{y}^\star)$ under the distribution.

$$\min_{p_{\boldsymbol{y}}} \mathcal{J}, \quad \text{where } \mathcal{J} \triangleq \mathbb{E}_{\boldsymbol{x}\sim p_{\boldsymbol{y}}(\cdot)}\left[f(\boldsymbol{x}, \boldsymbol{y}^\star)\right]. \tag{10}$$

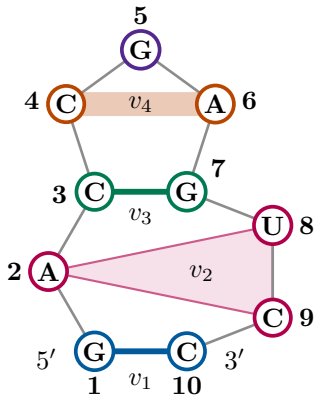
They approximate this expected value with an average over a set of sampled sequences $\mathcal{S}$.

$$\mathcal{J} \approx \frac{1}{|\mathcal{S}|} \sum_{\boldsymbol{x}\in\mathcal{S}} f(\boldsymbol{x}, \boldsymbol{y}^\star). \tag{11}$$

If the distribution puts all of its probability on one sequence $\boldsymbol{x}$, then $\mathcal{J}$ equals $f(\boldsymbol{x}, \boldsymbol{y}^\star)$, so this formula optimizes for that sequence only. The discrete problem in Eq. 9 is therefore a special case of optimizing over a distribution.

3.2 Coupled-variable distributions

To build this distribution, we turn the target structure into variables. Each variable is a group of positions whose nucleotides are chosen together, which we describe as coupling. We couple positions in the same way as SamplingDesign (Tang et al., 2026), which creates the four types of variables in Fig. 1, namely unpaired positions, paired positions, mismatches, and trimismatches, with 4, 6, 16, and 64 choices. Paired positions are coupled so that every sampled pair is valid, and mismatches and trimismatches are coupled because the energy model scores these unpaired nucleotides together. Each variable has its own local distribution, and the probability of a sequence is the product of these local distributions.



v_1	Pair	$(1, 10)$
v_2	Trimismatch	$(2, 8, 9)$
v_3	Pair	$(3, 7)$
v_4	Mismatch	$(4, 6)$
v_5	Unpaired	(5)

Figure 1: Example of a target structure being split into variables. Each color is one variable, and the positions in a variable are chosen together.

$$\begin{aligned} p_{\boldsymbol{y}}(\boldsymbol{x}) \triangleq & \prod_{i\in \mathit{unpaired}''(\boldsymbol{y}^\star)} p^{\mathrm{u}}_i(x_i) \\ & \cdot \prod_{(i,j)\in \mathit{pairs}(\boldsymbol{y}^\star)} p^{\mathrm{p}}_{i,j}(x_i x_j) \\ & \cdot \prod_{(i,j)\in \mathit{mismatches}(\boldsymbol{y}^\star)} p^{\mathrm{m}}_{i,j}(x_i x_j) \\ & \cdot \prod_{(i,j,k)\in \mathit{trimismatches}(\boldsymbol{y}^\star)} p^{\mathrm{tm}}_{i,j,k}(x_i x_j x_k). \end{aligned} \tag{12}$$

Here $\mathit{unpaired}''(\boldsymbol{y}^\star)$ holds the unpaired positions that are not in a mismatch or trimismatch. Any sequence drawn from this distribution is in $\mathcal{X}(\boldsymbol{y}^\star)$.

3.3 SamplingDesign and its runtime bottleneck

SamplingDesign showed that optimizing this distribution produces high-quality designs, but this quality comes at a high computational cost, which is evident from the way it updates the distribution. SamplingDesign uses gradient descent, estimating the gradient of $\mathcal{J}$ from the sampled sequences at each step (Tang et al., 2026).

$$\nabla_{p_{\boldsymbol{y}}} \mathcal{J}^{(t)} \approx \frac{1}{|\mathcal{S}|} \sum_{\boldsymbol{x}\in\mathcal{S}} \nabla_{p_{\boldsymbol{y}}} \log p_{\boldsymbol{y}}(\boldsymbol{x})\, f(\boldsymbol{x}, \boldsymbol{y}^\star). \tag{13}$$

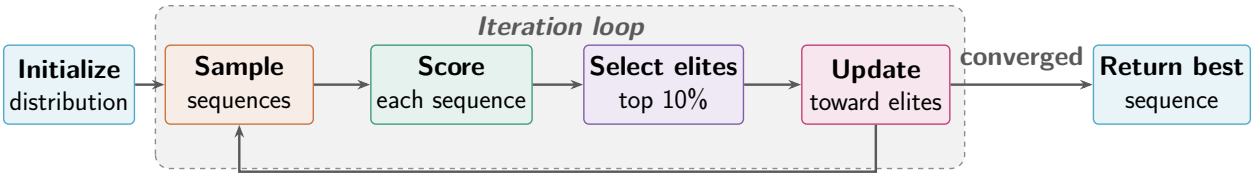


Figure 2: Overview of EliteDesign, which follows the cross-entropy method. Each round samples sequences from the current distribution, scores them, and moves the distribution toward the best ones.

This estimated gradient is an average over all samples, so every sample must be scored at each step ($|\mathcal{S}| = 2500$ by default). Scoring a sample means running LinearPartition (Zhang et al., 2020) on it, and this accounts for over 99% of SamplingDesign's runtime (Tang et al., 2026), because scoring a large number of sequences is expensive even though LinearPartition itself is efficient. The gradient also only gives a direction, so the distribution moves a small amount at each step (a learning rate of 0.01 with up to 2,000 steps by default). As a result, SamplingDesign takes a long time to run, especially on longer puzzles, where a single step can take over 20 seconds (Tang et al., 2026).

In other words, the runtime of SamplingDesign depends heavily on the number of sequences scored, rather than on computing the gradient itself. This motivates the question of whether a distribution can learn from fewer samples and larger updates, possibly without a gradient at all.

4 EliteDesign

4.1 The cross-entropy method

The cross-entropy method (de Boer et al., 2005; Rubinstein, 1999) is a gradient-free optimization method that was initially developed for rare-event simulation and later applied to optimization problems. It repeats two steps, sampling candidates from a distribution and then moving the distribution toward the best candidates.

For the second step, let $\mathcal{E}$ be the set of elites, which are the best sequences in the current sample, and give each elite $\boldsymbol{x} \in \mathcal{E}$ a weight $w_{\text{elite}}(\boldsymbol{x})$. The method aims to form the distribution p_{elites} under which the weighted elites are most likely.

$$p_{\text{elites}} = \operatorname*{argmax}_{p} \sum_{\boldsymbol{x} \in \mathcal{E}} w_{\text{elite}}(\boldsymbol{x}) \log p(\boldsymbol{x}). \tag{14}$$

Here p is any distribution of the form in Eq. 12, and $p(\boldsymbol{x})$ is the probability that it gives to the sequence $\boldsymbol{x}$. Maximizing this sum is the same as minimizing the cross-entropy between the elite weights and p.

$$p_{\text{elites}} = \operatorname*{argmin}_{p} H(w_{\text{elite}}, p). \tag{15}$$

Here H is the cross-entropy, which is defined as

$$H(w_{\text{elite}}, p) \triangleq -\sum_{\boldsymbol{x} \in \mathcal{E}} w_{\text{elite}}(\boldsymbol{x}) \log p(\boldsymbol{x}). \tag{16}$$

So p_{elites} is the distribution with the lowest cross-entropy to the weighted elites, which gives the method its name. In the basic method, all elites count equally and p_{elites} directly becomes the next distribution. However, the cross-entropy method can also work with weighted elites and smoothed updates, as discussed in de Boer et al. (2005). We apply the method in this way, by weighing the elites (Sec. 4.4) and smoothing the update (Sec. 4.5).

The cross-entropy method is a good candidate for the RNA design problem because this update is simple. Our distribution is a product of independent local distributions (Eq. 12), so for each variable, the new probability of a choice is the summed weight of the elites that made it. No gradient is needed, and one update can make a large shift to the distribution.

4.2 Objective functions

In EliteDesign, we take advantage of the fact that the cross-entropy method can apply more than one objective function. We use one objective, f_{r}, to rank the samples and select the elites, and a second objective, f_{w}, to weigh the elites. This allows us to optimize for two different objectives at the same time, where higher is better for both.

$$f(\boldsymbol{x}, \boldsymbol{y}^\star) \triangleq \begin{cases} f_{\text{r}}(\boldsymbol{x}, \boldsymbol{y}^\star) & \text{to rank} \\ f_{\text{w}}(\boldsymbol{x}, \boldsymbol{y}^\star) & \text{to weigh} \end{cases} \tag{17}$$

The two objectives are interchangeable, so f_{r} and f_{w} can each be based on any of the metrics in Sec. 2, or on any other function of a sequence, since neither needs to be differentiable. In this work, we rank by

the log of the Boltzmann probability and weigh by one minus NED.

$$f(\boldsymbol{x}, \boldsymbol{y}^\star) = \begin{cases} f_{\mathrm{r}} = \log p(\boldsymbol{y}^\star \mid \boldsymbol{x}) & \text{to rank} \\ f_{\mathrm{w}} = 1 - \mathrm{NED}(\boldsymbol{x}, \boldsymbol{y}^\star) & \text{to weigh} \end{cases} \tag{18}$$

We found that optimizing both objectives together, rather than Boltzmann probability alone or NED alone, works better in practice. Boltzmann probability decides which sequences the distribution learns from, while NED makes the distribution learn more from elites with a lower ensemble defect. This produces a new distribution that favors sequences with a high Boltzmann probability and a low ensemble defect.

4.3 Initialization and sampling

To initialize the sampling distribution of our coupled variables, we favor `A` at unpaired positions and `CG` or `GC` at paired positions, and we initialize mismatch and trimismatch variables by multiplying the unpaired probabilities of their bases.

At each round t, EliteDesign samples N sequences from the current distribution $p^{(t)}_{\mathrm{current}}$ with a small amount of uniform mixing, so that every variable choice remains possible.

$$p_{\mathrm{sampling}} = (1 - \delta)\, p^{(t)}_{\mathrm{current}} + \delta U. \tag{19}$$

In this equation, U is the uniform distribution for a variable and δ is the uniform mixing rate. The mixing is only used for sampling and does not change the stored distribution. We score each sequence in the sample with LinearPartition (Zhang et al., 2020), which gives both the Boltzmann probability and NED, cache the scores to avoid scoring the same sequence twice, and track the best sequence found so far.

4.4 Selection of elites

We sort the sequences in a sample by f_{r}, which in this work ranks them by Boltzmann probability, select the top 10% as the elites $\mathcal{E}$, and give each elite $\boldsymbol{x}$ a weight based on f_{w},

$$w_{\mathrm{elite}}(\boldsymbol{x}) = \frac{f_{\mathrm{w}}(\boldsymbol{x}, \boldsymbol{y}^\star)}{\sum_{\boldsymbol{x}' \in \mathcal{E}} f_{\mathrm{w}}(\boldsymbol{x}', \boldsymbol{y}^\star)}, \tag{20}$$

where the sum in the denominator runs over every elite $\boldsymbol{x}'$ in $\mathcal{E}$, so the weights add up to 1. We use $f_{\mathrm{w}}(\boldsymbol{x}, \boldsymbol{y}^\star) = 1 - \mathrm{NED}(\boldsymbol{x}, \boldsymbol{y}^\star)$, so that elites with a lower NED influence the distribution update more.

4.5 Distribution updates and stopping criteria

After weighing the elites, we get the distribution p_{elites} of Eq. 14, which we then combine with the current distribution using the smoothed cross-entropy update (de Boer et al., 2005).

$$p^{(t+1)}_{\mathrm{current}} = (1 - \alpha)\, p^{(t)}_{\mathrm{current}} + \alpha\, p_{\mathrm{elites}}. \tag{21}$$

Here α is the elite update rate, which is set to 0.5 by default in our implementation, so each round moves the distribution halfway to the elites. The main goal of these updates is to move the distribution toward a region in which we are likely to find the best sequence.

At each round, we track two quantities, the highest value of the ranking objective f_{r} seen so far, and the mean value of f_{r} over the sample, averaged over the last h rounds. We call h the patience. We stop when both of the following conditions hold, or when a limit on rounds or scored sequences is reached.

1. The highest value of f_{r} has not improved for h rounds.
2. The moving average of f_{r} has not improved for h rounds.

4.6 Returning the best sequence

The final output of EliteDesign is a single sequence, which is the best one it has seen during the search. Writing $\mathcal{S}^{(t)}$ for the sequences scored in round t, this is

$$\boldsymbol{x}^\star = \operatorname*{argmax}_{\boldsymbol{x} \in \bigcup_t \mathcal{S}^{(t)}} f_{\mathrm{r}}(\boldsymbol{x}, \boldsymbol{y}^\star). \tag{22}$$

Here we return the best sequence under f_{r}, the objective used for ranking, but the method can also return the best sequence found for any other metric, such as NED or structural distance. Because LinearPartition only approximates the Boltzmann probability, we check this choice after the search is done. We rescore $\boldsymbol{x}^\star$, the sequence with the best NED, and the most probable sequence under the final distribution exactly with ViennaRNA (Lorenz et al., 2011), and we return the one with the highest Boltzmann probability.

5 Results

5.1 Evaluation setup

We test EliteDesign on two datasets, Eterna100 and 16S rRNA. On Eterna100, we compare EliteDesign to SamplingDesign (Tang et al., 2026),

Method	N	Mean $p(\boldsymbol{y}^\star \mid \boldsymbol{x})$ ↑	Geom. mean $p(\boldsymbol{y}^\star \mid \boldsymbol{x})$ ↑	NED ↓	Struct. dist. ↓	Energy gap ↓	MFE/ uMFE ↑	Time per structure (s) ↓
EliteDesign	100	0.6032	0.5601	0.03145	1.62	0.525	78/74	**4.21**
EliteDesign	500	**0.6105**	**0.5825**	**0.03015**	**1.50**	**0.5024**	**81**/**77**	11.99
SamplingDesign	2500	0.5950	0.5225	0.03518	2.15	0.72	80/76	2133
SAMFEO	-	0.581	0.167	0.043	4.57	2.48	77/74	—
NEMO	-	0.271	0.083	0.098	1.93	1.31	79/**77**	—
Probability ceiling	-	0.7406	0.8545	—	—	—	—	—

Table 1: Results on Eterna100. As in SamplingDesign (Tang et al., 2026), we report the best value of each metric across five seeds and all sequences sampled during optimization, so MFE and uMFE count the puzzles where any sampled sequence is a solution. The geometric mean excludes the 18 proven undesignable puzzles. The scores for SamplingDesign, SAMFEO, and NEMO are from Tang et al. (2026). The times compare SamplingDesign with EliteDesign run with minimal bookkeeping, tracking only the best $p(\boldsymbol{y}^\star \mid \boldsymbol{x})$ and the best NED.

SAMFEO (Zhou et al., 2023), and NEMO (Portela, 2018), and on 16S rRNA, we compare it to SAMFEO. We run EliteDesign with the following settings.

1. Population sizes of $N = 100$ and $N = 500$, with uniform mixing of 0.2% and elite update rate $\alpha = 0.5$ (see Table 6 in the appendix).
2. Five random seeds per target; for each metric, we select the best seed.
3. We use LinearPartition (Zhang et al., 2020) for scoring during search and ViennaRNA (Lorenz et al., 2011) for final scoring.
4. As in SamplingDesign (Tang et al., 2026), we report the best value of each metric over all sampled sequences.

Algorithm 1 EliteDesign

function DESIGN($\boldsymbol{y}^\star$, f_r, f_w)
  $p^{(1)}_\mathrm{current} \leftarrow \mathrm{init}(\boldsymbol{y}^\star)$; $\boldsymbol{x}^\star \leftarrow$ None
  for $t = 1, 2, \ldots$ until stopped **do**
    $\mathcal{S} \leftarrow N$ samples from $(1-\delta)\, p^{(t)}_\mathrm{current} + \delta U$
    score $\mathcal{S}$ with f_r and f_w
    update $\boldsymbol{x}^\star$ if any sequence has higher f_r
    $\mathcal{E} \leftarrow$ the top 10% of $\mathcal{S}$ by f_r
    compute w_elite on $\mathcal{E}$ ▷ Eq. 20
    update $p^{(t+1)}_\mathrm{current}$ toward p_elites ▷ Eq. 21
    score the most probable sequence
    update $\boldsymbol{x}^\star$ if it has higher f_r
  end for
  rescore the final candidates with ViennaRNA
  return $\boldsymbol{x}^\star$
end function

To keep the runtime comparison fair, we ran both EliteDesign and SamplingDesign on a 96-core AMD CPU. On the 16S targets, EliteDesign and SAMFEO were both run on 96-core CPUs, and SAMFEO's times are from Zhou et al. (2026b).

We also report an upper bound on Boltzmann probability from LinearDecompose (Zhou et al., 2026a), which we call the probability ceiling. It bounds the Boltzmann probability of any sequence, so it shows how much room may be left on each target.

5.2 Eterna100

The Eterna100 dataset is a benchmark of 100 puzzles from the Eterna game, assembled by Anderson-Lee and colleagues (Anderson-Lee et al., 2016). It contains mostly synthetic RNA secondary structures that range from 12 to 400 nucleotides and vary in design difficulty, allowing us to compare methods across diverse targets.

Table 1 shows that EliteDesign with $N = 500$ gives the best scores on all quality metrics, and ties NEMO for uMFE solutions, while running 178× faster than SamplingDesign. Even with $N = 100$, EliteDesign beats SamplingDesign on all of these metrics except MFE and uMFE solutions, while running about 500× faster.

Fig. 3 compares EliteDesign with $N = 500$ and SamplingDesign on individual puzzles. EliteDesign finds a higher Boltzmann probability on 65 puzzles and a lower one on 13. The other 22 puzzles lie on an axis of the plot, because their difference in Boltzmann probability or NED is at most 10^{-5}. The majority of this gain comes from long puzzles. On the 50 longest puzzles, EliteDesign wins on 44 and

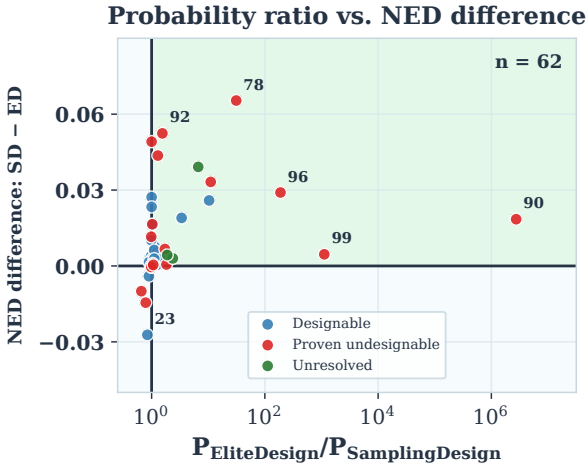


Figure 3: EliteDesign ($N = 500$) versus SamplingDesign on individual Eterna100 puzzles. The x-axis is the ratio of their Boltzmann probabilities, and the y-axis is SamplingDesign's NED minus EliteDesign's NED. The top right corner shows that EliteDesign is better than SamplingDesign on both metrics for 62 puzzles.

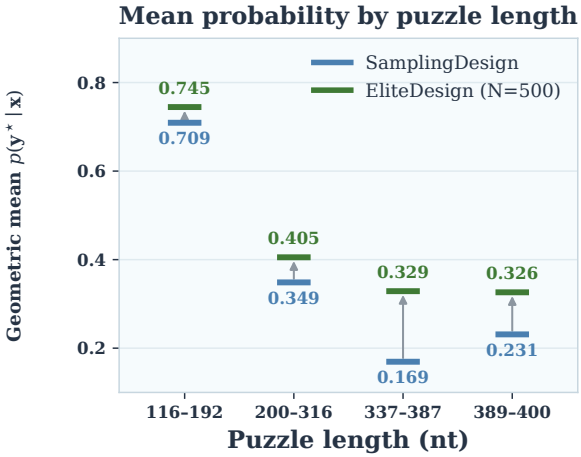


Figure 4: Geometric mean Boltzmann probability for the 40 longest Eterna100 puzzles, in four groups of 10 by length. Proven undesignable puzzles are removed from each group, as in SamplingDesign (Tang et al., 2026). The gain from EliteDesign ($N = 500$) is largest on long puzzles.

loses on none, while on the 50 shortest it wins on 21 and loses on 13, by small margins.

Fig. 4 shows that the geometric mean gain grows with puzzle length, reaching almost twice SamplingDesign's value on the puzzles of 337 to 387 nucleotides. EliteDesign also finds higher Boltzmann probabilities on other datasets (Tables 3–5 in the appendix).

5.3 Example: Puzzle 90

Puzzle `#90` ("Gladius", 400 nucleotides) is one of the hardest puzzles in Eterna100. It is proven undesignable, so no sequence can have its target structure $\boldsymbol{y}^\star$ as an MFE structure. Fig. 5 compares the sequences with the highest Boltzmann probability returned by EliteDesign and SamplingDesign. It shows that EliteDesign's Boltzmann probability is about 2.7 million times higher than that of SamplingDesign (1.31×10^{-11} versus 4.77×10^{-18}). We also see that its MFE structure has a structural distance of 20 from $\boldsymbol{y}^\star$, compared with 42 for SamplingDesign, which corresponds to EliteDesign missing 10 target pairs and SamplingDesign missing 21. Its free energy gap is also lower, at 7.9 kcal/mol compared with 19.7 kcal/mol. This improvement is also shown very clearly in the base-pairing probability plots, where we see that SamplingDesign's sequence has much more competition from incorrect pairs.

5.4 16S rRNA

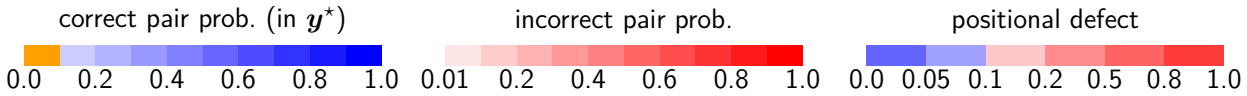
To test scalability, we evaluate EliteDesign on ten 16S rRNA targets from 950 to 1,525 nucleotides. Since SamplingDesign is far too slow for targets of this length, we compare against SAMFEO, the strongest scalable baseline. Table 2 and Fig. 6 show that EliteDesign finds a higher Boltzmann probability on every target, with a mean of 0.52 versus 0.20, about 2.6 times higher. It also lowers mean NED from 0.0036 to 0.0012. Fig. 7 shows that EliteDesign with $N = 500$ achieves this dominance over SAMFEO while taking 66 minutes per target on average, compared with 946 minutes for SAMFEO, making it 14× faster. Even when the sample size is reduced to $N = 100$, EliteDesign achieves a 24× speedup over SAMFEO, with only a small drop in quality from $N = 500$.

6 Discussion

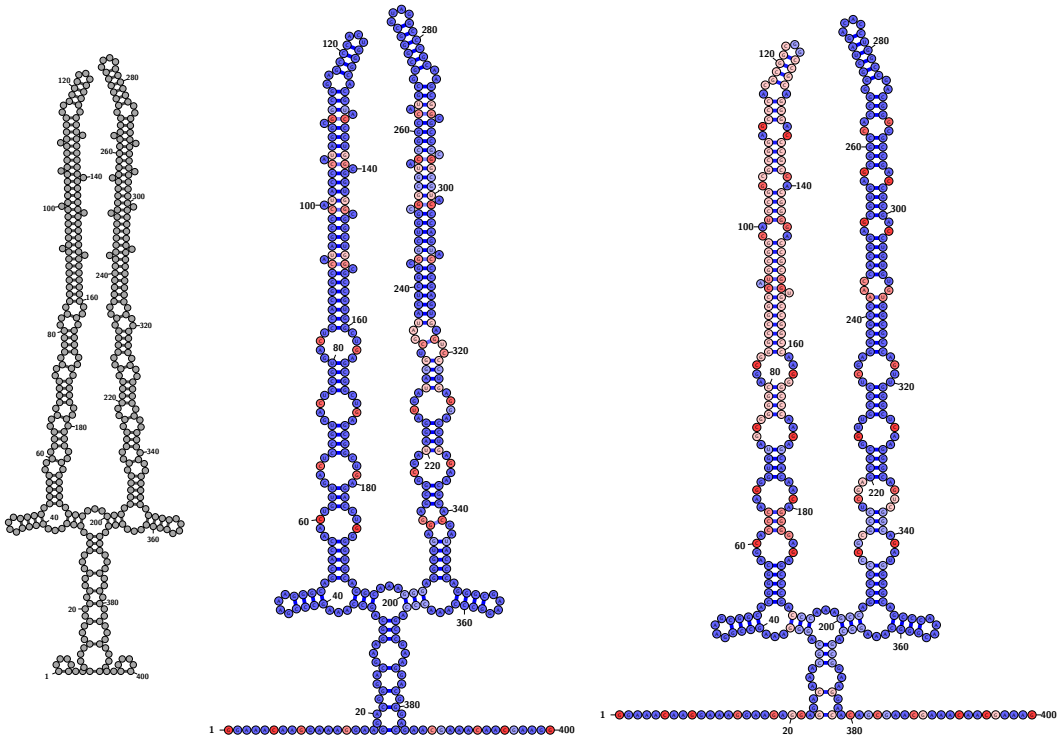
In this work, we introduced EliteDesign, an RNA design framework that uses the gradient-free cross-entropy method to find high-probability RNA sequences for a given target structure. Our method beats the state-of-the-art SamplingDesign on Eterna100 in both speed and quality, and we show large gains over SAMFEO on ten 16S rRNA targets while remaining much faster per puzzle. We further show that this method addresses a gap in current

(a)

#90 (400 nt) "Gladius"	Boltzmann prob. (↑)	NED (↓)	struct. dist. (↓)	energy gap (kcal/mol) (↓)	MFE	uMFE
EliteDesign ($N = 500$)	$\mathbf{1.31 \times 10^{-11}}$	**0.081**	**20**	**7.9**	No	No
SamplingDesign	4.77×10^{-18}	0.134	42	19.7	No	No
Probability ceiling	1.71×10^{-8}	—	—	—	—	—



(b) Target $\boldsymbol{y}^\star$ (c) MFE of EliteDesign ($N = 500$) (d) MFE of SamplingDesign



(e) Base-pairing probabilities, EliteDesign ($N = 500$) (f) Base-pairing probabilities, SamplingDesign

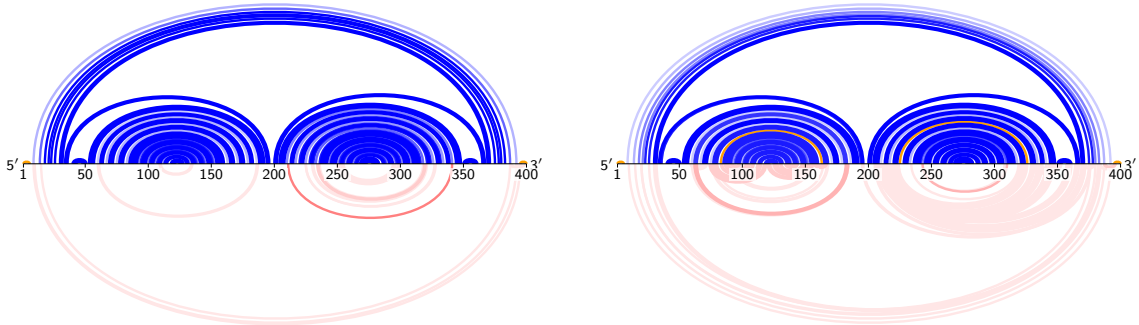


Figure 5: This figure shows the sequences with the highest Boltzmann probability from EliteDesign ($N = 500$) and SamplingDesign on Puzzle #90, which is a proven undesignable puzzle. (a) The scores of each design and a probability upper bound on $p(\boldsymbol{y}^\star \mid \boldsymbol{x})$ (Zhou et al., 2026a). (b) The target structure $\boldsymbol{y}^\star$. (c)–(d) The MFE structure of each design. Blue bonds belong to $\boldsymbol{y}^\star$ and red bonds do not, darker bonds have higher probability, and nucleotides are colored by positional defect. (e)–(f) Base-pairing probabilities, where blue arcs are the pairs that match between a structure $\boldsymbol{y}'$ in the ensemble and $\boldsymbol{y}^\star$, orange arcs are matching pairs with a probability under 0.1, and red arcs are the pairs of $\boldsymbol{y}'$ that do not match $\boldsymbol{y}^\star$. Darker arcs have higher probability.

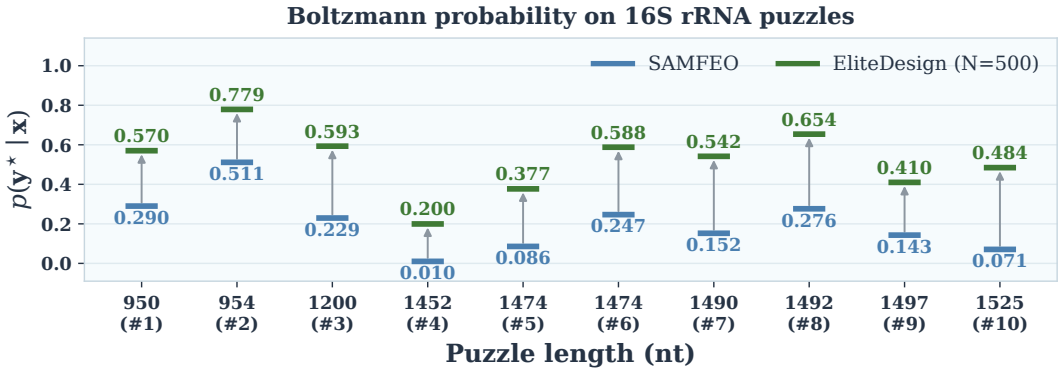


Figure 6: Boltzmann probability on ten 16S rRNA targets. EliteDesign ($N = 500$) achieves a better Boltzmann probability than SAMFEO on every target.

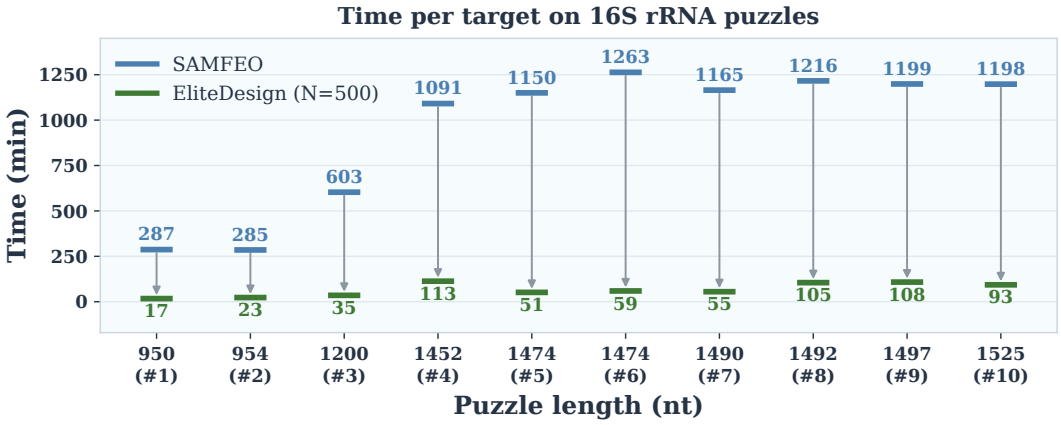


Figure 7: Time per target in minutes on the same 16S rRNA targets. EliteDesign ($N = 500$) is faster than SAMFEO on every target.

		Boltzmann probability $p(\boldsymbol{y}^\star \mid \boldsymbol{x})$ ↑			**NED** ↓			**Time (min)** ↓		
Target	**nt**	SAMFEO	EliteDesign (N=100)	EliteDesign (N=500)	SAMFEO	EliteDesign (N=100)	EliteDesign (N=500)	SAMFEO	EliteDesign (N=100)	EliteDesign (N=500)
1	950	0.290	**0.575**	0.570	0.0026	0.0013	**0.0012**	287	**9.8**	17.1
2	954	0.511	0.773	**0.779**	0.0018	**0.0006**	**0.0006**	285	**8.6**	22.8
3	1200	0.229	0.587	**0.593**	0.0028	0.0010	**0.0009**	603	**8.8**	35.0
4	1452	0.010	0.192	**0.200**	0.0079	0.0028	**0.0027**	1,091	**48.5**	113
5	1474	0.086	0.344	**0.377**	0.0040	0.0016	**0.0014**	1,150	**43.1**	51.4
6	1474	0.247	0.553	**0.588**	0.0024	0.0010	**0.0009**	1,263	**48.6**	59.2
7	1490	0.152	0.540	**0.542**	0.0038	**0.0010**	**0.0010**	1,165	**30.1**	55.1
8	1492	0.276	0.652	**0.654**	0.0023	**0.0007**	**0.0007**	1,216	**68.0**	105
9	1497	0.143	0.406	**0.410**	0.0038	**0.0017**	**0.0017**	1,199	**62.0**	108
10	1525	0.071	0.474	**0.484**	0.0044	**0.0012**	**0.0012**	1,198	**67.0**	93.0
Mean		0.202	0.510	**0.520**	0.0036	0.0013	**0.0012**	946	**39.4**	66.0

Table 2: Results on ten 16S rRNA targets. N is the number of sequences sampled per round.

RNA design methods by enabling the design of sequences longer than 1,000 nucleotides with relatively high Boltzmann probability compared with other methods. We also show that our method achieves higher Boltzmann probability on other RNA design datasets such as Rfam (Kalvari et al., 2021; Taneda, 2011) and RNAsolo (Adamczyk et al., 2022).

6.1 Why EliteDesign works

EliteDesign keeps the backbone of SamplingDesign by searching for a high-probability sequence with a distribution over the design space. However, it changes how this distribution learns. SamplingDesign averages a gradient over all of its samples and takes a small step, whereas EliteDesign focuses on learning from the best samples and moves the distribution strongly toward them in one update. As a result, it needs far fewer scoring calls to LinearPartition. For example, on Eterna100 with $N = 500$, EliteDesign samples about 61,000 sequences on average and takes about 120 rounds, whereas SamplingDesign samples about 4.35 million sequences and takes about 1,740 rounds.

EliteDesign also takes advantage of optimizing for two objectives, as shown in Eq. 17, where either objective can be replaced without changing the algorithm. Since neither needs a gradient, even an objective that is not differentiable can be used.

6.2 Limitations

Despite these gains, EliteDesign has some limitations.

1. EliteDesign uses the same initialization strategy for every target, which may not work equally well for all structures. During optimization, it can concentrate too early on a narrow set of sequences and miss better solutions. This may explain the small losses to SamplingDesign on some short puzzles.
2. EliteDesign uses LinearPartition to estimate Boltzmann probabilities during search. Errors in these estimates can cause it to favor weaker candidates and miss better ones. Exact scoring at the end checks the final candidates but cannot recover sequences missed during search.
3. The probability ceiling shows that room remains on many targets, for example on Snowflake 4, where EliteDesign reaches 47% of the ceiling. However, a large gap can come either from the design or from a loose bound, so it does not prove that a better sequence exists.

6.3 Ablation study

TODO.

6.4 Future work

We see several directions for extending this work.

1. Future work could add a local refinement stage similar to SAMFEO (Zhou et al., 2023), which might further improve NED and Boltzmann probability on some structures.
2. Because the objectives are interchangeable, EliteDesign can be applied to other objectives and to other design problems such as mRNA design (Zhang et al., 2023).
3. While EliteDesign does well on computational benchmarks, where designs are only scored with a thermodynamic model, the designs still need to be tested in the laboratory.

6.5 Code availability

Our code is available at `https://github.com/JSONFN21/EliteDesign`.

References

B. Adamczyk, M. Antczak, and M. Szachniuk. RNAsolo: a repository of cleaned PDB-derived RNA 3D structures. *Bioinformatics*, 38(14):3668–3670, 2022. 10.1093/bioinformatics/btac386.

J. Anderson-Lee, E. Fisker, V. Kosaraju, M. Wu, J. Kong, J. Lee, M. Lee, M. Zada, A. Treuille, R. Das, and Eterna Players. Principles for predicting RNA secondary structure design difficulty. *Journal of Molecular Biology*, 428(5):748–757, 2016. 10.1016/j.jmb.2015.11.013.

R. R. Breaker. Riboswitches and the RNA world. *Cold Spring Harbor Perspectives in Biology*, 4(2): a003566, 2012. 10.1101/cshperspect.a003566.

P.-T. de Boer, D. P. Kroese, S. Mannor, and R. Y. Rubinstein. A tutorial on the cross-entropy method. *Annals of Operations Research*, 134:19–67, 2005. 10.1007/s10479-005-5724-z.

R. M. Dirks, M. Lin, E. Winfree, and N. A. Pierce. Paradigms for computational nucleic acid design. *Nucleic Acids Research*, 32(4):1392–1403, 2004. 10.1093/nar/gkh291.

J. A. Doudna and T. R. Cech. The chemical repertoire of natural ribozymes. *Nature*, 418:222–228, 2002. 10.1038/418222a.

I. L. Hofacker, W. Fontana, P. F. Stadler, L. S. Bonhoeffer, M. Tacker, and P. Schuster. Fast folding and comparison of RNA secondary structures. *Monatshefte für Chemie*, 125(2):167–188, 1994. 10.1007/BF00818163.

I. Kalvari, E. P. Nawrocki, N. Ontiveros-Palacios, J. Argasinska, K. Lamkiewicz, M. Marz, S. Griffiths-Jones, C. Toffano-Nioche, D. Gautheret, Z. Weinberg, E. Rivas, S. R. Eddy, R. D. Finn, A. Bateman, and A. I. Petrov. Rfam 14: expanded coverage of metagenomic, viral and microRNA families. *Nucleic Acids Research*, 49(D1): D192–D200, 2021. 10.1093/nar/gkaa1047.

R. Lorenz, S. H. Bernhart, C. Höner zu Siederdissen, H. Tafer, C. Flamm, P. F. Stadler, and I. L. Hofacker. ViennaRNA package 2.0. *Algorithms for Molecular Biology*, 6:26, 2011. 10.1186/1748-7188-6-26.

D. H. Mathews, M. D. Disney, J. L. Childs, S. J. Schroeder, M. Zuker, and D. H. Turner. Incorporating chemical modification constraints into a dynamic programming algorithm for prediction of RNA secondary structure. *Proceedings of the National Academy of Sciences*, 101(19):7287–7292, 2004. 10.1073/pnas.0401799101.

M. Matthies, R. Krueger, A. Torda, and M. Ward. Differentiable partition function calculation for RNA. *Nucleic Acids Research*, 2023.

J. S. McCaskill. The equilibrium partition function and base pair binding probabilities for RNA secondary structure. *Biopolymers*, 29(6–7):1105–1119, 1990. 10.1002/bip.360290621.

F. Portela. An unexpectedly effective Monte Carlo technique for the RNA inverse folding problem. *bioRxiv*, page 345587, 2018. 10.1101/345587.

L. A. Rojas et al. Personalized RNA neoantigen vaccines stimulate T cells in pancreatic cancer. *Nature*, 618(7963):144–150, 2023. 10.1038/s41586-023-06063-y.

R. Rubinstein. The cross-entropy method for combinatorial and continuous optimization. *Methodology and Computing in Applied Probability*, 1: 127–190, 1999. 10.1023/A:1010091220143.

A. Taneda. MODENA: a multi-objective RNA inverse folding. *Advances and Applications in Bioinformatics and Chemistry*, 4:1, 2011.

W. Y. Tang, N. Dai, T. Zhou, D. H. Mathews, and L. Huang. SamplingDesign: RNA design via continuous optimization with coupled variables and Monte-Carlo sampling. *Nature Communications*, 17:2950, 2026. 10.1038/s41467-025-67901-3.

J. N. Zadeh, B. R. Wolfe, and N. A. Pierce. Nucleic acid sequence design via efficient ensemble defect optimization. *Journal of Computational Chemistry*, 32(3):439–452, 2011. 10.1002/jcc.21633.

H. Zhang, L. Zhang, D. H. Mathews, and L. Huang. LinearPartition: linear-time approximation of RNA folding partition function and base-pairing probabilities. *Bioinformatics*, 36(Supplement 1): i258–i267, 2020. 10.1093/bioinformatics/btaa460.

H. Zhang et al. Algorithm for optimized mRNA design improves stability and immunogenicity. *Nature*, 621(7978):396–403, 2023. 10.1038/s41586-023-06127-z.

T. Zhou, N. Dai, S. Li, M. Ward, D. H. Mathews, and L. Huang. RNA design via structure-aware multifrontier ensemble optimization. *Bioinformatics*, 39(Supplement 1):i563–i571, 2023. 10.1093/bioinformatics/btad252.

T. Zhou, D. H. Mathews, and L. Huang. Probabilistic RNA designability via interpretable ensemble approximation and dynamic decomposition. *Bioinformatics*, 42(Supplement 1):btag246, 2026a. 10.1093/bioinformatics/btag246.

T. Zhou, D. H. Mathews, and L. Huang. Fast and versatile RNA design via motif-level divide-and-conquer and structure-level rival search. *arXiv preprint arXiv:2603.02283*, 2026b.

A Supplementary Information

A.1 Other datasets

Table 3: Eterna100-V2

Method	Mean $p(y^\star \mid x)$ ↑	Geom. mean $p(y^\star \mid x)$ ↑	NED ↓	Structural distance ↓	Mean energy gap $\Delta\Delta G^\circ$ ↓	MFE/ uMFE ↑
EliteDesign ($N = 100$)	0.6262	0.3507	0.02520	0.41	0.0497	90/84
EliteDesign ($N = 500$)	**0.6353**	**0.3677**	**0.02422**	**0.31**	**0.0377**	**94**/88
SamplingDesign	0.617	0.264	0.029	0.93	0.19	89/84
SAMFEO	0.596	0.059	0.036	1.94	0.61	83/79
NEMO	0.280	0.022	0.095	0.94	0.49	91/**89**

Table 4: Rfam27

Method	Mean $p(y^\star \mid x)$ ↑	NED ↓
EliteDesign ($N = 100$)	0.8188	**0.003616**
EliteDesign ($N = 500$)	**0.8194**	0.003682
SamplingDesign	0.8115	0.003974
SAMFEO	0.8091	0.003923
NEMO	0.4448	0.02577

Table 5: RNAsolo (longest 100)

Method	Mean $p(y^\star \mid x)$ ↑	NED ↓
EliteDesign ($N = 100$)	0.4942	0.01342
EliteDesign ($N = 500$)	**0.4982**	**0.01314**
SAMFEO	0.4866	0.01480
NEMO	0.1807	0.04655

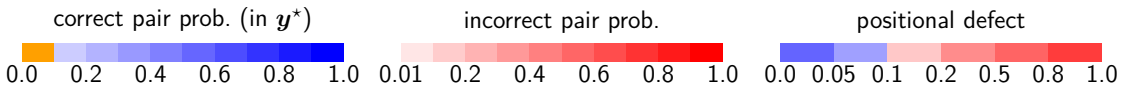
A.2 Default settings

Table 6

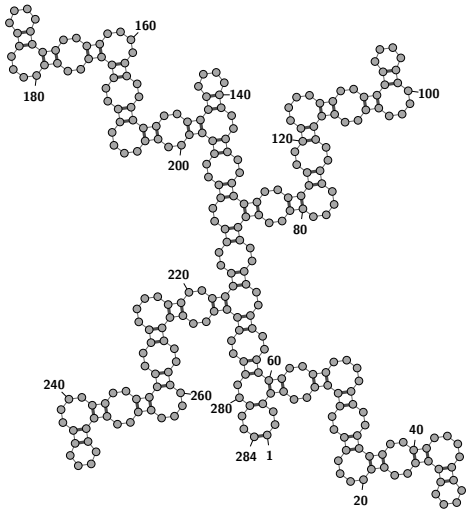
Setting	Symbol	Default
Samples per round	N	500
Elite fraction	—	10%
Elite update rate	α	0.5
Uniform mixing	δ	0.002
Initialization bias	ϵ	0.75
Patience	h	25
LinearPartition beam size	b	250

A.3 Other examples

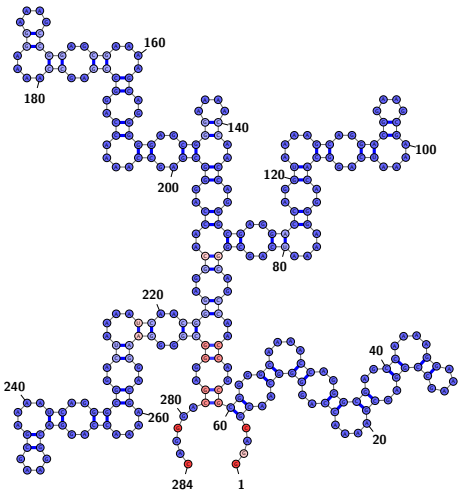
	#78 (284 nt) "Mat - Lot 2-2 B"	Boltzmann prob. $p(\boldsymbol{y}^\star \mid \boldsymbol{x})(\uparrow)$	ensemble defect NED $(\boldsymbol{x}, \boldsymbol{y}^\star)(\downarrow)$	struct. distance $d(\text{MFE}(\boldsymbol{x}), \boldsymbol{y}^\star)(\downarrow)$	energy gap $\Delta\Delta G^\circ\,(\boldsymbol{x}, \boldsymbol{y}^\star)(\downarrow)$	is MFE	is uMFE
(a)	EliteDesign ($N = 500$)	**0.0073**	**0.042**	**4**	**2.4 kcal/mol**	No	No
	SamplingDesign	2.35×10^{-4}	0.157	**4**	3.3 kcal/mol	No	No
	Probability ceiling	0.0184	—	—	—	—	—



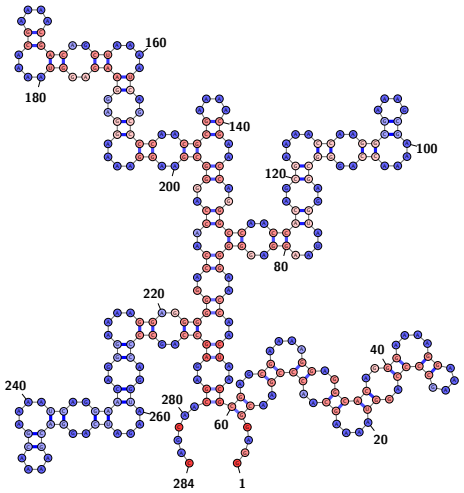
(b) Target structure $\boldsymbol{y}^\star$



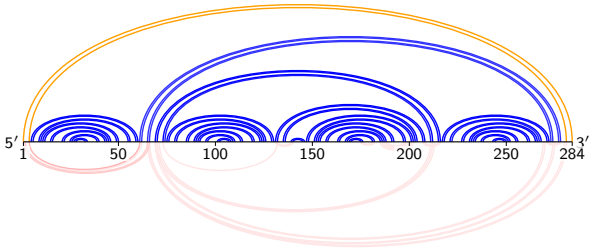
(c) MFE structure of EliteDesign ($N = 500$)



(d) MFE structure of SamplingDesign



(e) Base-pairing probabilities, EliteDesign ($N = 500$)



(f) Base-pairing probabilities, SamplingDesign

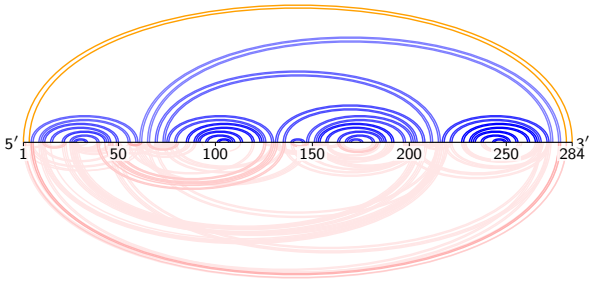
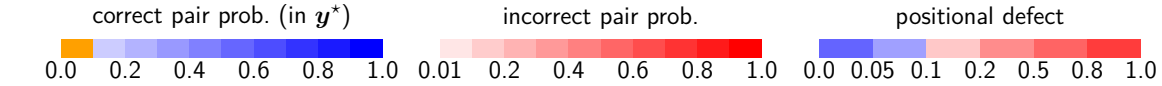


Figure 8

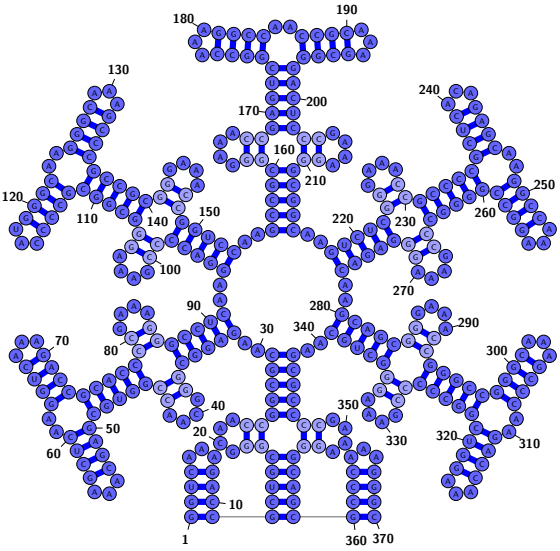
(a)

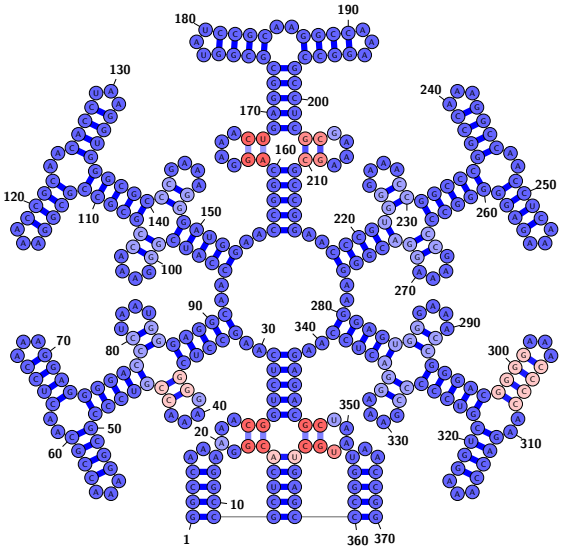
#73 (370 nt) "Snowflake 4"	Boltzmann prob. $p(\boldsymbol{y}^\star \mid \boldsymbol{x})(\uparrow)$	ensemble defect NED $(\boldsymbol{x}, \boldsymbol{y}^\star)(\downarrow)$	struct. distance $d(\mathrm{MFE}(\boldsymbol{x}), \boldsymbol{y}^\star)(\downarrow)$	energy gap $\Delta\Delta G^\circ(\boldsymbol{x}, \boldsymbol{y}^\star)(\downarrow)$	is MFE	is uMFE
EliteDesign ($N = 500$)	**0.3459**	**0.0122**	**0**	**0.0 kcal/mol**	**Yes**	**Yes**
SamplingDesign	0.0334	0.0418	**0**	**0.0 kcal/mol**	**Yes**	**Yes**
Probability ceiling	0.7341	—	—	—	—	—



(b) MFE structure of EliteDesign ($N = 500$) ($= \boldsymbol{y}^\star$)

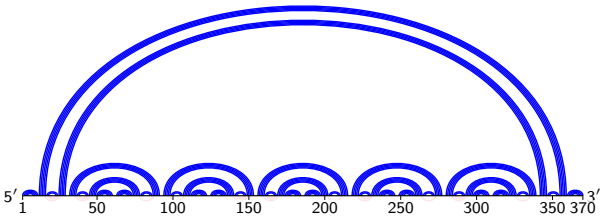
(c) MFE structure of SamplingDesign ($= \boldsymbol{y}^\star$)





(d) Base-pairing probabilities, EliteDesign ($N = 500$)

(e) Base-pairing probabilities, SamplingDesign



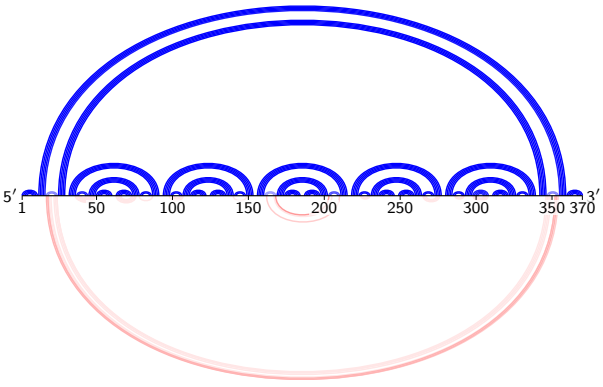
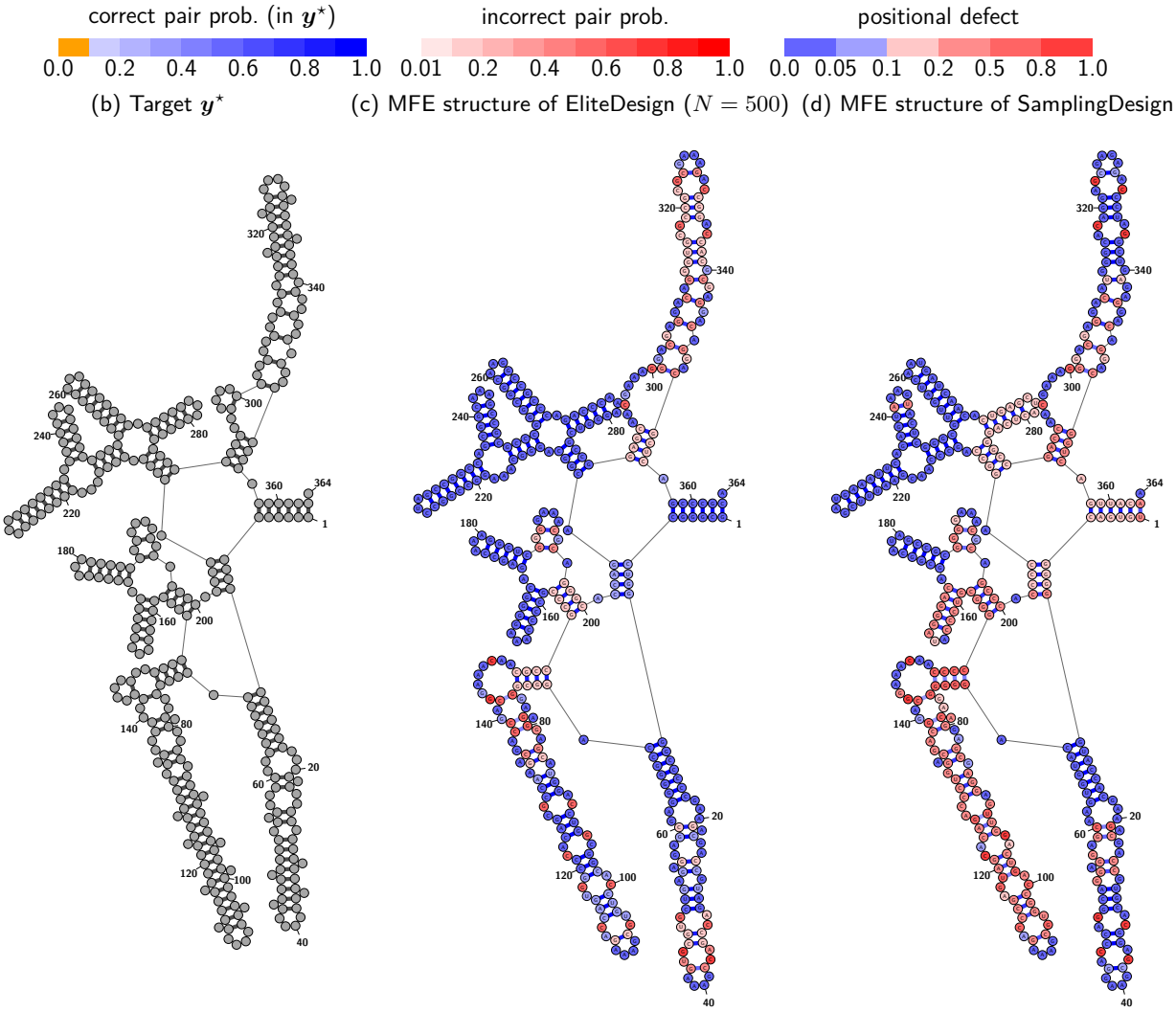


Figure 9

(a)

#99 (364 nt) "Shooting Star"	Boltzmann prob. $p(\boldsymbol{y}^\star \mid \boldsymbol{x})(\uparrow)$	ensemble defect NED $(\boldsymbol{x}, \boldsymbol{y}^\star)(\downarrow)$	struct. distance $d(\text{MFE}(\boldsymbol{x}), \boldsymbol{y}^\star)(\downarrow)$	energy gap $\Delta\Delta G^\circ\,(\boldsymbol{x}, \boldsymbol{y}^\star)(\downarrow)$	is MFE	is uMFE
EliteDesign ($N = 500$)	$\mathbf{6.16 \times 10^{-7}}$	**0.107**	**20**	**3.8 kcal/mol**	No	No
SamplingDesign	5.50×10^{-10}	0.173	**20**	7.8 kcal/mol	No	No
Probability ceiling	5.15×10^{-4}	—	—	—	—	—



(b) Target $\boldsymbol{y}^\star$ (c) MFE structure of EliteDesign ($N = 500$) (d) MFE structure of SamplingDesign

(e) Base-pairing probabilities, EliteDesign ($N = 500$) (f) Base-pairing probabilities, SamplingDesign

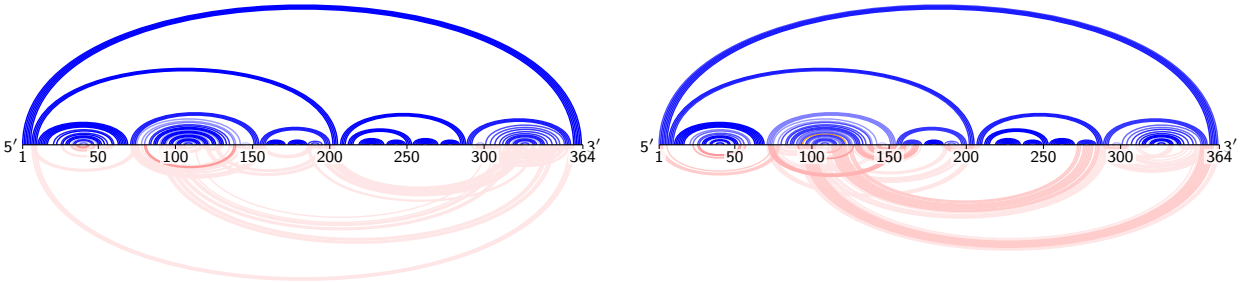


Figure 10

(a)

#86 (355 nt) "Methaqualone"	Boltzmann prob. $p(\boldsymbol{y}^\star \mid \boldsymbol{x})(\uparrow)$	ensemble defect NED $(\boldsymbol{x}, \boldsymbol{y}^\star)(\downarrow)$	struct. distance $d(\text{MFE}(\boldsymbol{x}), \boldsymbol{y}^\star)(\downarrow)$	energy gap $\Delta\Delta G^\circ\,(\boldsymbol{x}, \boldsymbol{y}^\star)(\downarrow)$	is MFE	is uMFE
EliteDesign ($N = 500$)	$\mathbf{4.28 \times 10^{-6}}$	**0.066**	**12**	**6.1 kcal/mol**	No	No
SamplingDesign	3.87×10^{-7}	0.112	**12**	6.5 kcal/mol	No	No
Probability ceiling	6.95×10^{-5}	—	—	—	—	—

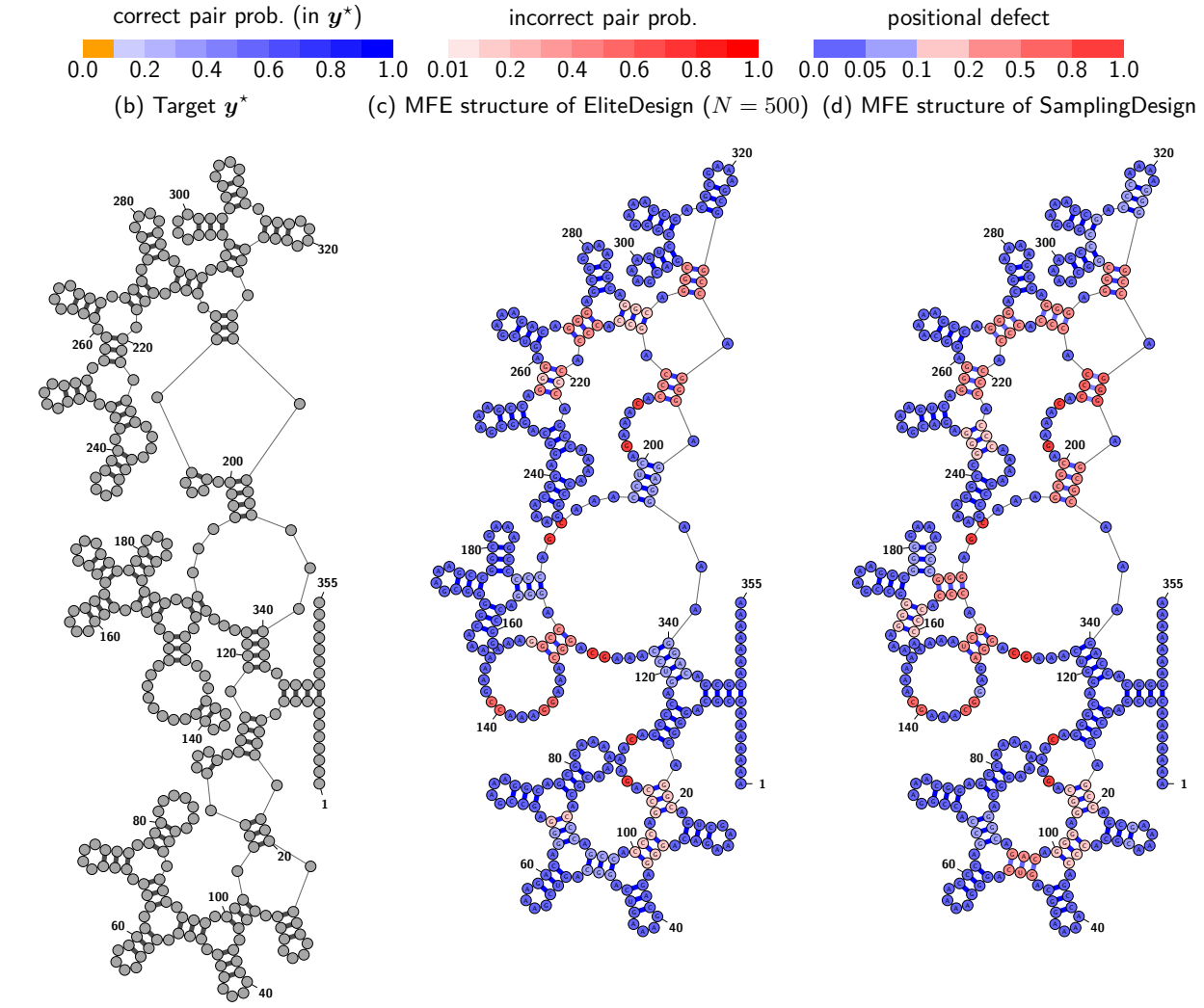
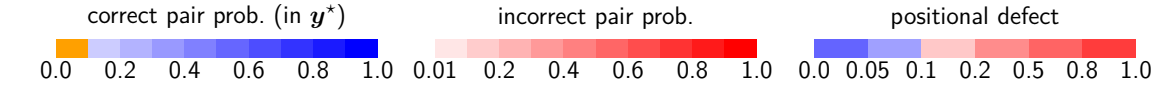


Figure 11

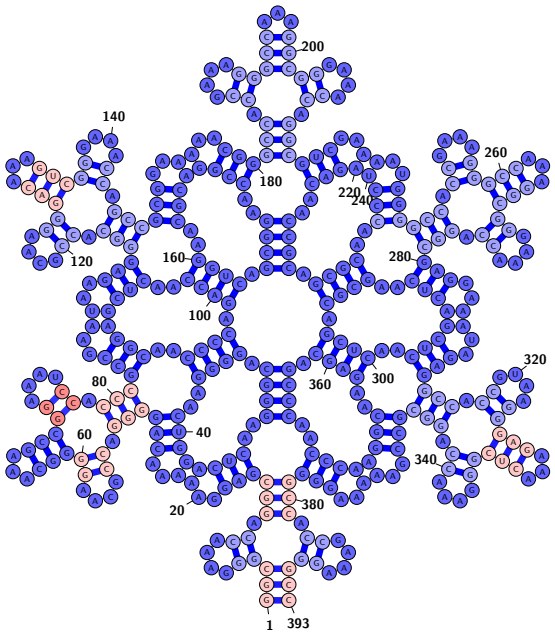
(a)

#76 (393 nt) "Snowflake 3"	Boltzmann prob. $p(\boldsymbol{y}^\star \mid \boldsymbol{x})(\uparrow)$	ensemble defect NED $(\boldsymbol{x}, \boldsymbol{y}^\star)(\downarrow)$	struct. distance $d(\mathrm{MFE}(\boldsymbol{x}), \boldsymbol{y}^\star)(\downarrow)$	energy gap $\Delta\Delta G^\circ\,(\boldsymbol{x}, \boldsymbol{y}^\star)(\downarrow)$	is MFE	is uMFE
EliteDesign ($N = 500$)	**0.1504**	**0.0355**	**0**	**0.0 kcal/mol**	**Yes**	**Yes**
SamplingDesign	0.0444	0.0531	**0**	**0.0 kcal/mol**	**Yes**	**Yes**
Probability ceiling	0.7611	—	—	—	—	—

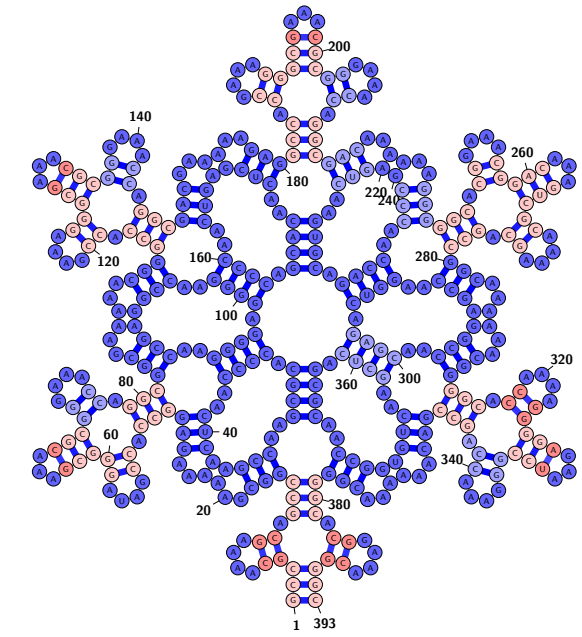


(b) MFE structure of EliteDesign ($N = 500$) ($= \boldsymbol{y}^\star$)

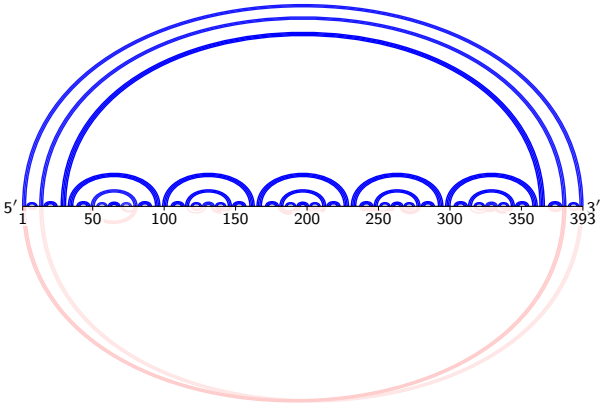
(c) MFE structure of SamplingDesign ($= \boldsymbol{y}^\star$)



(d) Base-pairing probabilities, EliteDesign ($N = 500$)



(e) Base-pairing probabilities, SamplingDesign



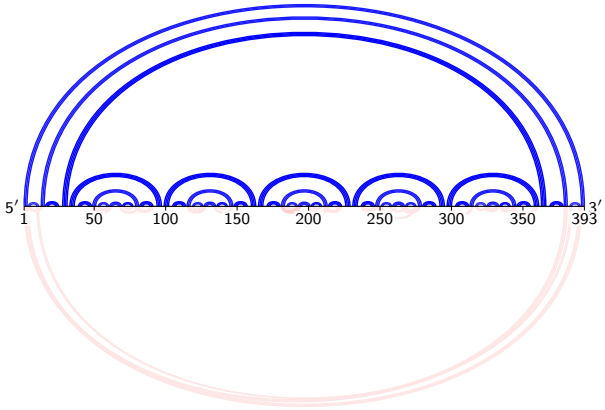
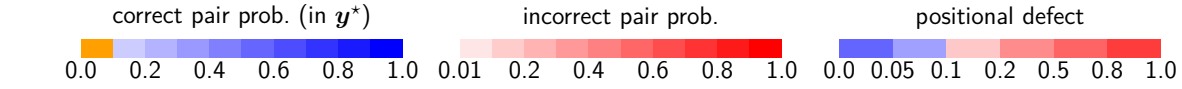


Figure 12

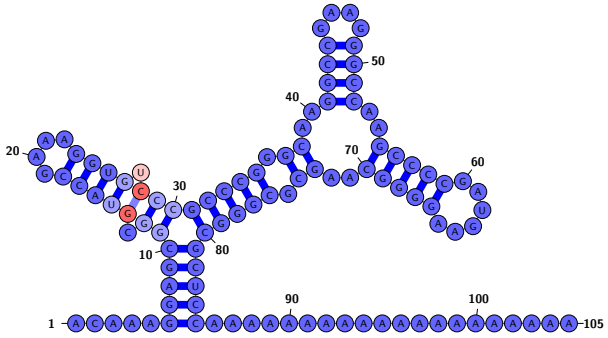
(a)

#77 (105 nt) "Hard Y and a bit more"	Boltzmann prob. $p(\boldsymbol{y}^\star \mid \boldsymbol{x})(\uparrow)$	ensemble defect NED $(\boldsymbol{x}, \boldsymbol{y}^\star)(\downarrow)$	struct. distance $d(\text{MFE}(\boldsymbol{x}), \boldsymbol{y}^\star)(\downarrow)$	energy gap $\Delta\Delta G^\circ\,(\boldsymbol{x}, \boldsymbol{y}^\star)(\downarrow)$	is MFE	is uMFE
EliteDesign ($N = 500$)	**0.4523**	**0.0178**	**0**	**0.0 kcal/mol**	**Yes**	**Yes**
SamplingDesign	0.3664	0.0195	3	**0.0 kcal/mol**	**Yes**	No
Probability ceiling	0.5037	—	—	—	—	—

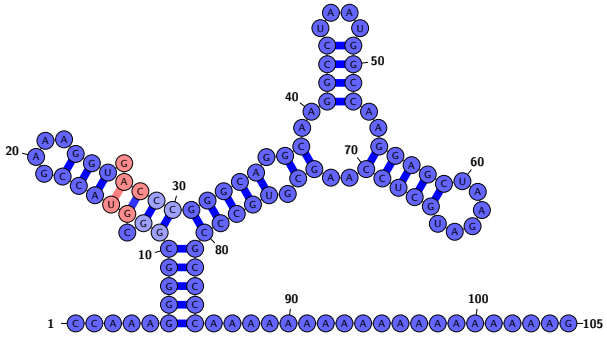


(b) MFE structure of EliteDesign ($N = 500$) ($= \boldsymbol{y}^\star$)

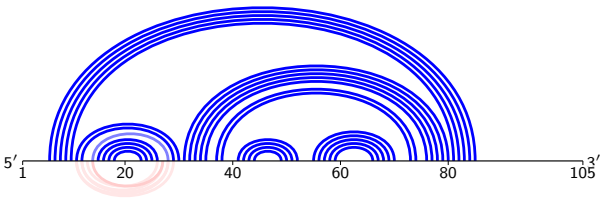
(c) MFE structure of SamplingDesign



(d) Base-pairing probabilities, EliteDesign ($N = 500$)



(e) Base-pairing probabilities, SamplingDesign



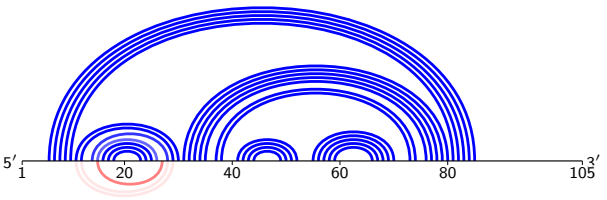
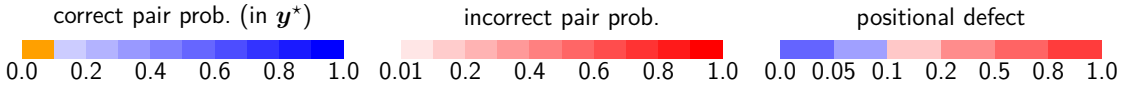


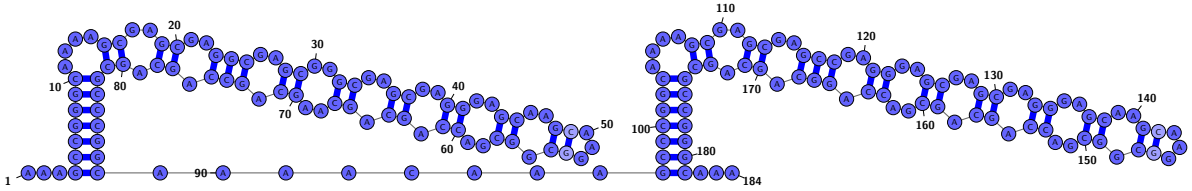
Figure 13

(a)

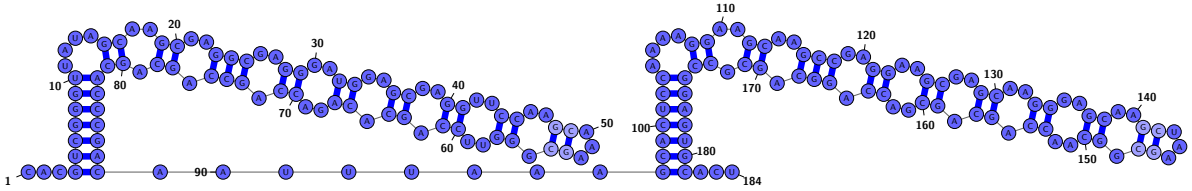
#70 (184 nt) "Pokeball"	Boltzmann prob. $p(\boldsymbol{y}^\star \mid \boldsymbol{x})(\uparrow)$	ensemble defect NED $(\boldsymbol{x}, \boldsymbol{y}^\star)(\downarrow)$	struct. distance $d(\text{MFE}(\boldsymbol{x}), \boldsymbol{y}^\star)(\downarrow)$	energy gap $\Delta\Delta G^\circ\,(\boldsymbol{x}, \boldsymbol{y}^\star)(\downarrow)$	is MFE	is uMFE
EliteDesign ($N = 500$)	**0.8428**	**0.0030**	**0**	**0.0 kcal/mol**	**Yes**	**Yes**
SamplingDesign	0.6802	0.0115	**0**	**0.0 kcal/mol**	**Yes**	**Yes**
Probability ceiling	0.8890	—	—	—	—	—



(b) MFE structure of EliteDesign ($N = 500$) ($= \boldsymbol{y}^\star$)



(c) MFE structure of SamplingDesign ($= \boldsymbol{y}^\star$)



(d) Base-pairing probabilities, EliteDesign ($N = 500$)

(e) Base-pairing probabilities, SamplingDesign

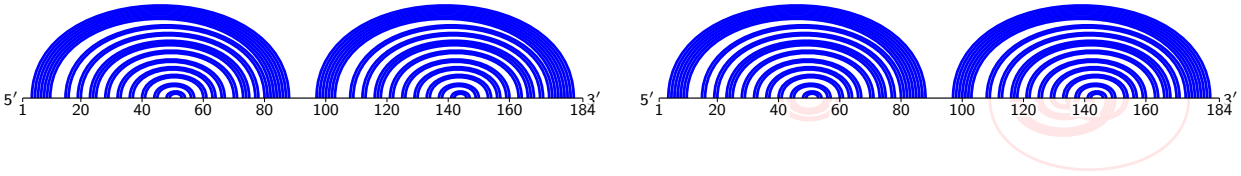


Figure 14